Sprint 10 · Redes neuronales bien entrenadas

¿una red neuronal mejora al aprendizaje automático clásico y a qué precio?

FECHAS
13–17 de noviembre

HORAS
15 h (5 h teoría · 6 h práctica · 4 h PBL) · **Módulo 4, UD 4.1–4.3**

Qué aprenderás

1. Cómo funciona por dentro una red neuronal: neurona, capas, activación, propagación hacia delante y hacia atrás.
2. Construir y entrenar un MLP con Keras para clasificar cancelaciones y predecir el precio por noche.
3. Reconocer el **sobreajuste** en las curvas de aprendizaje y combatirlo (L2, *dropout*, parada temprana, *batch norm*).
4. Ajustar tasa de aprendizaje y tamaño de lote, usar *callbacks* y guardar/cargar modelos.
5. Decidir con datos si una red "vale la pena" frente a un modelo de boosting.

Requisitos previos

- Sprints 7 y 8: partición entrenamiento/validación/prueba, métricas (AUC, F1, MAE) y modelos de árboles/boosting.
- Python y pandas del Bloque B. No hace falta saber matemáticas avanzadas: se explican con ejemplos.
- Datos: `reservas_turisdata.csv` (se descarga solo desde el notebook). Todo funciona **sin internet** y en Colab.

Prácticas

Notebook	Tema	Tiempo
`S10_01_red_desde_cero`	Una red neuronal con solo NumPy: neurona, activación, *backprop*, entrenamiento	1 h 15
`S10_02_mlp_keras`	MLP en Keras: cancelaciones (clasificación) y `precio_noche` (regresión)	1 h 30
`S10_03_regularizacion`	Curvas, sobreajuste, L2, *dropout*, parada temprana, lote, tasa, *callbacks*, guardar modelo	1 h 45
`S10_04_reto_vale_la_pena`	Esqueleto del reto: MLP frente a boosting con incertidumbre y diagnóstico	1 h 30 (+ trabajo PBL)

Cada notebook tiene ejercicios con comprobaciones automáticas (`assert`): si saltan, el mensaje te dice qué revisar.

RETO Reto PBL · «¿Vale la pena la red?»

Contexto del cliente. La dirección de TurisData Canarias oye hablar de redes neuronales en todas partes y pregunta si debe sustituir su modelo de cancelaciones por una. No quiere una opinión: quiere evidencias, coste incluido, para decidir si merece la pena el cambio.

Qué se entrega y en qué formato

1. **Notebook** ejecutable de principio a fin (basado en `S10_04`), con semillas fijas.
2. **Informe comparativo** de 1–2 páginas (PDF o Markdown) *ML clásico frente a MLP*: tabla de resultados, curvas de aprendizaje y recomendación.

Restricciones

- Un solo problema a elegir: cancelaciones (`cancelada`) **o** precio (`precio_noche`). El otro es opcional.
- Misma partición para todos los modelos; la prueba se usa **una sola vez**.
- Modelo de referencia: tu campeón del Sprint 8 (o el boosting que trae el notebook).
- Máximo 40 épocas por entrenamiento y modelos pequeños (se ejecuta en CPU en pocos minutos).
- Sin fugas de datos: el escalador se ajusta solo con el entrenamiento.

Criterios de evaluación (0–4, la media alimenta "ejercicios y entregas")

Criterio	Qué buscamos aquí
Corrección técnica	Partición correcta, preprocesado sin fuga, MLP regularizado, comparación justa (varias semillas o intervalo), curvas bien interpretadas.

Criterio	Qué buscamos aquí
Reproducibilidad	Se ejecuta desde el README; semillas y versiones fijadas; datos y script incluidos.
Análisis y comunicación al cliente	Respuesta clara a "¿vale la pena?": métricas con incertidumbre, coste (tiempo, complejidad, explicabilidad) y recomendación accionable con sus límites.
IA responsable	Riesgos concretos: datos sintéticos, umbral y coste de errores (overbooking), posibles sesgos por país o canal; medidas aplicadas y evaluadas.

Pasos sugeridos

- **Día 1:** ejecuta `S10_04`, entiende cada bloque y elige el problema. Reproduce tu modelo de referencia.
- **Día 2:** entrena el MLP, prueba 2–3 variantes (tamaño, *dropout*, L2, tasa) mirando **solo la validación**.
- **Día 3:** diagnostica el sobreajuste con curvas y compara con la referencia (varias semillas, intervalo de confianza).
- **Día 4:** redacta el informe y prepara la presentación de 5 minutos.

Qué se enseña en la *review*. La tabla comparativa, una curva de aprendizaje comentada y la recomendación en una frase ("Recomendamos… porque…"), con el coste y los riesgos.

Definition of Done

- [ ] El notebook se ejecuta de arriba abajo sin errores y con semillas fijas.
- [ ] La partición entrenamiento/validación/prueba es única y la prueba se usa una sola vez.
- [ ] Hay al menos un MLP y un modelo de árboles/boosting comparados con las mismas métricas.
- [ ] Incluye curvas de aprendizaje y un diagnóstico explícito de sobreajuste.
- [ ] La comparación tiene en cuenta la incertidumbre (varias semillas o bootstrap).
- [ ] El informe recomienda algo concreto, con coste y límites.
- [ ] Se mencionan al menos dos riesgos de IA responsable específicos de este caso.

Recursos

- Documentación de Keras: guía «Sequential model» y «Training & evaluation with the built-in methods» (keras.io).
- Documentación de scikit-learn: `HistGradientBoostingClassifier` y métricas (`roc_auc_score`, `average_precision_score`).
- Vídeos/artículos sobre curvas de aprendizaje y sobreajuste (busca «learning curves overfitting» y elige uno con ejemplos).
- Guía de callbacks de Keras: `EarlyStopping`, `ReduceLROnPlateau`, `ModelCheckpoint`.
- Apuntes del curso: UD 4.1–4.3 (teoría de este sprint).